



New Mansoura University
Faculty of Computer Science and Engineering



Advanced Machine Learning

Clustering Applications - DBSCAN

Islam Reda

Associate Professor

islam-cis@mans.edu.eg



Outline

- Using clustering for Color Segmentation
- Using Clustering for Preprocessing
- Using Clustering for Semi-Supervised Learning
- DBSCAN



Using Clustering for Image Segmentation



Using Clustering for Image Segmentation

- **Image segmentation** is the task of partitioning an image into multiple segments.
- In **semantic segmentation**, all pixels that are part of the same object type get assigned to the same segment. For example, in a self-driving car's vision system, all pixels that are part of a pedestrian's image might be assigned to the "pedestrian" segment.
- In **instance segmentation**, all pixels that are part of the same individual object are assigned to the same segment. In this case there would be a different segment for each pedestrian.



Using Clustering for Image Segmentation

- The state of the art in semantic or instance segmentation today is achieved using complex architectures based on convolutional neural networks.
- Here, we are going to do something much simpler: **color segmentation**. We will simply assign pixels to the same segment if they have a similar color. In some applications, this may be sufficient, for example if you want to analyze satellite images to measure how much total forest area there is in a region, color segmentation may be just fine.



Using Clustering for Image Segmentation

- First, let's load the image using Matplotlib's **imread()** function:

```
>>> from matplotlib.image import imread # you could also use `imageio.imread()`  
>>> image = imread(os.path.join("images", "clustering", "ladybug.png"))  
>>> image.shape  
(533, 800, 3)
```

- The image is represented as a 3D array: the first dimension's size is the height, the second is the width, and the third is the number of color channels, in this case red, green and blue (RGB). The intensities of red, green and blue, each between 0.0 and 1.0 (or between 0 and 255 if you use **imageio.imread()**).



Using Clustering for Image Segmentation

- The following code reshapes the array to get a long list of RGB colors, then it clusters these colors using K-Means. Next, for each color (e.g., dark green), it looks for the mean color of the pixel's color cluster to replace the intensities of all cluster pixels with that mean. Finally it reshapes this long list of colors to get the same shape as the original image.

```
X = image.reshape(-1, 3)
kmeans = KMeans(n_clusters=8).fit(X)
segmented_img = kmeans.cluster_centers_[kmeans.labels_]
segmented_img = segmented_img.reshape(image.shape)
```



Original image

10 colors

8 colors



6 colors

4 colors

2 colors





Using Clustering for Preprocessing



Using Clustering for Preprocessing

- Clustering can be an efficient approach to dimensionality reduction, in particular as a preprocessing step before a supervised learning algorithm.
- For example, let's tackle the digits dataset which is a simple MNIST-like dataset containing 1,797 grayscale 8×8 images representing digits 0 to 9. First, let's load the dataset:

```
from sklearn.datasets import load_digits  
X_digits, y_digits = load_digits(return_X_y=True)
```

- Now, let's split it into a training set and a test set:

```
from sklearn.model_selection import train_test_split  
X_train, X_test, y_train, y_test = train_test_split(X_digits, y_digits)
```



Using Clustering for Preprocessing

- Next, let's fit a Logistic Regression model:

```
from sklearn.linear_model import LogisticRegression
log_reg = LogisticRegression(random_state=42)
log_reg.fit(X_train, y_train)
```

- Let's evaluate its accuracy on the test set:

```
>>> log_reg.score(X_test, y_test)
0.9666666666666667
```

- Okay, that's our **baseline: 96.7% accuracy**. Let's see if we can do better by using KMeans as a preprocessing step.
- We will create a pipeline that will first cluster the training set into 50 clusters and replace the images with their distances to these 50 clusters, then apply a logistic regression model.



Using Clustering for Preprocessing

```
from sklearn.pipeline import Pipeline
```

```
pipeline = Pipeline([  
    ("kmeans", KMeans(n_clusters=50)),  
    ("log_reg", LogisticRegression()),  
)  
pipeline.fit(X_train, y_train)
```

- Now let's evaluate this classification pipeline:

```
>>> pipeline.score(X_test, y_test)  
0.9822222222222222
```

- But we chose the number of clusters k completely arbitrarily, we can surely do better. Since K-Means is just a preprocessing step in a classification pipeline, the **best value of k** is simply the one that **results in the best classification performance** during cross-validation



Using Clustering for Preprocessing

- Let's use GridSearchCV to find the optimal number of clusters:

```
from sklearn.model_selection import GridSearchCV
```

```
param_grid = dict(kmeans__n_clusters=range(2, 100))
```

```
grid_clf = GridSearchCV(pipeline, param_grid, cv=3)
```

```
grid_clf.fit(X_train, y_train)
```

- Let's look at best value for k , and the performance of the resulting pipeline:

```
>>> grid_clf.best_params_
```

```
{'kmeans__n_clusters': 90}
```

```
>>> grid_clf.score(X_test, y_test)
```

```
0.9844444444444445
```

- With $k=90$ clusters, we get a small accuracy boost, reaching 98.4%.



Using Clustering for Semi-Supervised Learning



Using Clustering for Semi-Supervised Learning

- In semi-supervised learning, we have plenty of unlabeled instances and very few labeled instances.
- Let's train a logistic regression model on a sample of 50 labeled instances from the digits dataset:

```
n_labeled = 50
log_reg = LogisticRegression()
log_reg.fit(X_train[:n_labeled], y_train[:n_labeled])
```

- What is the performance of this model on the test set?

```
>>> log_reg.score(X_test, y_test)
0.8266666666666667
```

Using Clustering for Semi-Supervised Learning

- Let's see how we can do better. First, let's cluster the training set into 50 clusters, then for each cluster let's **find the image closest to the centroid**. We will call these images the **representative images**:

`k = 50`

```
kmeans = KMeans(n_clusters=k)
```

```
X_digits_dist = kmeans.fit_transform(X_train)
```

```
representative_digit_idx = np.argmin(X_digits_dist, axis=0)
```

```
X_representative_digits = X_train[representative_digit_idx]
```



Fifty representative digit images (one per cluster)



Using Clustering for Semi-Supervised Learning

- Now let's look at each image and manually label it:

```
y_representative_digits = np.array([4, 8, 0, 6, 8, 3, ..., 7, 6, 2, 3, 1, 1])
```

- Now we have a dataset with just 50 labeled instances, but instead of being completely random instances, each of them is a representative image of its cluster. Let's see if the performance is any better:

```
>>> log_reg = LogisticRegression()
>>> log_reg.fit(X_representative_digits, y_representative_digits)
>>> log_reg.score(X_test, y_test)
0.9244444444444444
```

- We jumped from 82.7% accuracy to 92.4%, although we are still only training the model on 50 instances.



Using Clustering for Semi-Supervised Learning

- We can go one step further: what if we propagated the labels to all the other instances in the same cluster? This is called **label propagation**:

```
y_train_propagated = np.empty(len(X_train), dtype=np.int32)
for i in range(k):
    y_train_propagated[kmeans.labels_==i] = y_representative_digits[i]
```

- Now let's train the model again and look at its performance:

```
>>> log_reg = LogisticRegression()
>>> log_reg.fit(X_train, y_train_propagated)
>>> log_reg.score(X_test, y_test)
0.9288888888888889
```

- We got a tiny little accuracy boost.



Using Clustering for Semi-Supervised Learning

- Let's see what happens if we only propagate the labels to the 20% of the instances that are closest to the centroids. Then, we train the model again on this partially propagated dataset:

```
>>> log_reg = LogisticRegression()  
>>> log_reg.fit(X_train_partially_propagated,  
y_train_partially_propagated)  
>>> log_reg.score(X_test, y_test)  
0.9422222222222222
```

- With just 50 labeled instances (only 5 examples per class on average!), we got 94.2% performance, which is pretty close to the performance of logistic regression on the fully labeled digits dataset (which was 96.7%).



DBSCAN

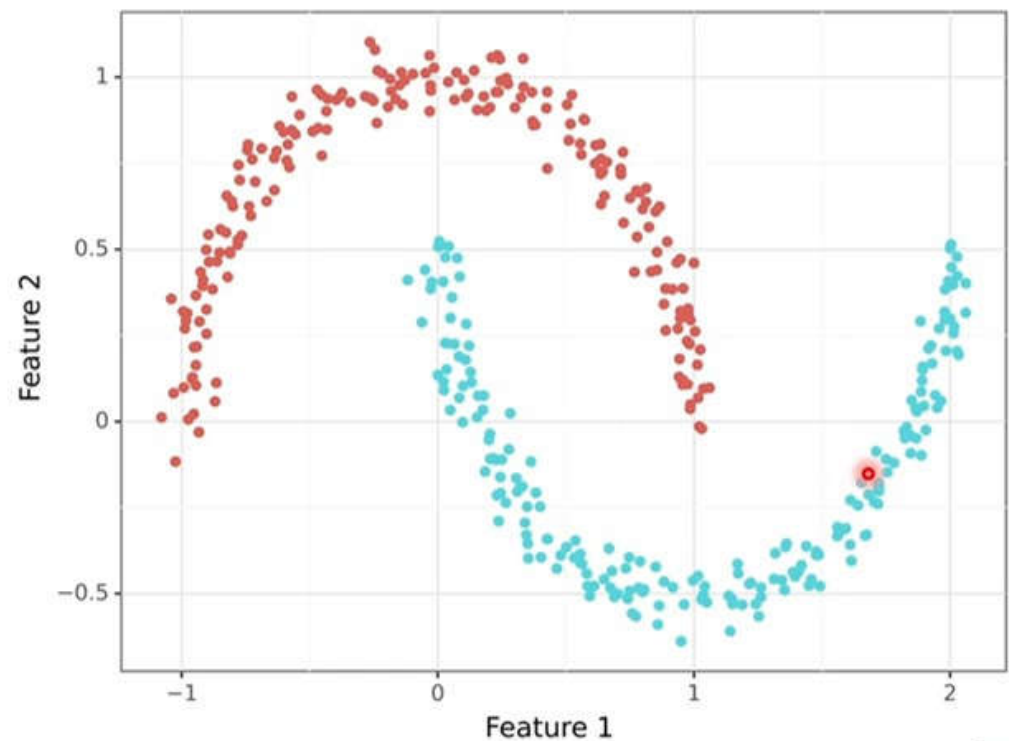


DBSCAN

- The **density-based spatial clustering of application with noise (DBSCAN)** algorithm is a density-based, partial partitioning clustering technique.
- Let's break that down:
 1. DBSCAN uses areas of low data point density vs. areas of high data point density to find clusters.
 2. The clusters produced by DBSCAN can be arbitrarily shaped.
 3. Not all data points are assigned to clusters – unassigned data points are called **noise points**.
 4. Non-noise data points will be assigned to a single cluster.

DBSCAN

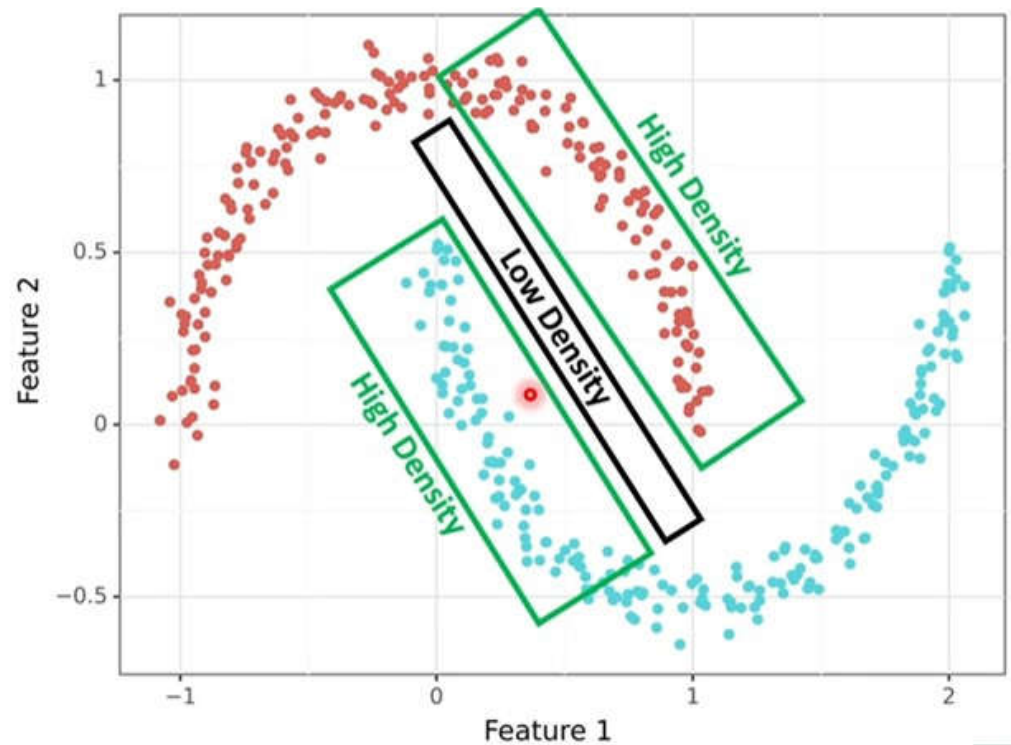
- **Density-based clusters** are regions dense with data points surrounded by regions with a low density of data points.



Density-based Clustering

DBSCAN

- Density-based clusters are used when clusters are irregular (e.g., not spherical), intertwined, and/or when outliers are present.
- DBSCAN can have problems identifying clusters if there are many areas of radically changing density.



Density-based Clustering



DBSCAN

- Types of points:
 - **Core points:** These are data points inside a cluster. A data point is core if it is surrounded by a minimum number of other data points within a set distance. When you use DBSCAN, you set the distance (known as **eps**) and the minimum number of points within the distance (known as **min_samples**). **A core point has many neighbors within distance, eps.**
 - **Border points:** A border point is not a core point, but falls within the set distance (i.e., **eps**) of a core point. **A border point: near core, but not dense enough to be core.**
 - **Noise points:** A noise point is any point that is neither a core point nor a border point.

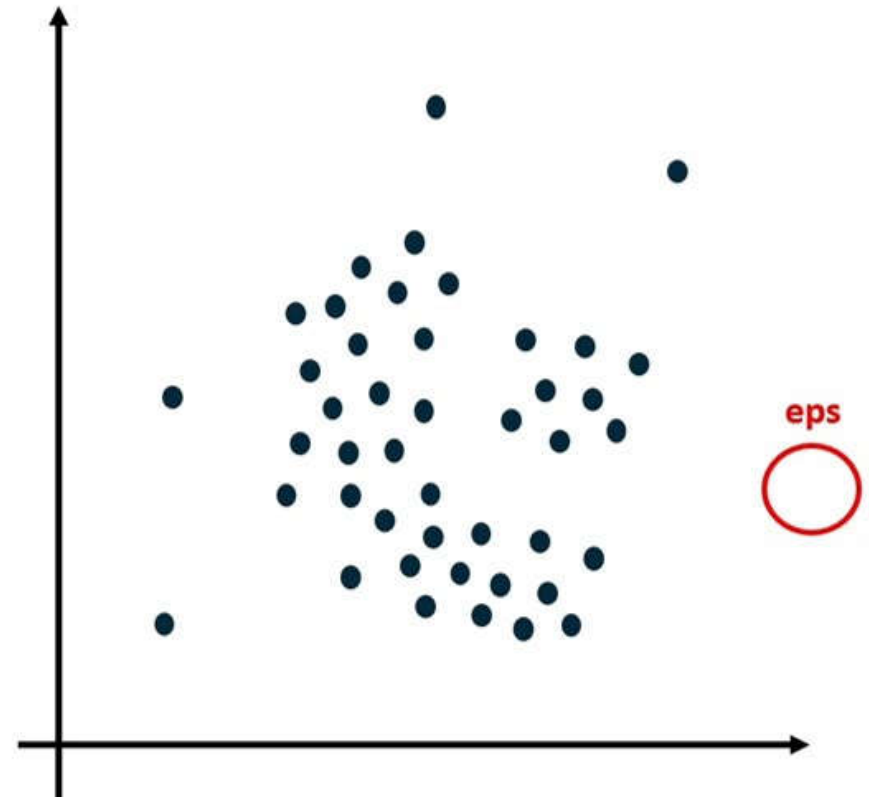


DBSCAN

- Algorithm steps:
 1. Mark each point as a core, border, or noise.
 2. Remove noise points (keep them marked as outliers (-1))
 3. Connect core points that are close to each other.
 4. Each connected core region becomes a cluster.
 5. Assign border points to nearest cluster.

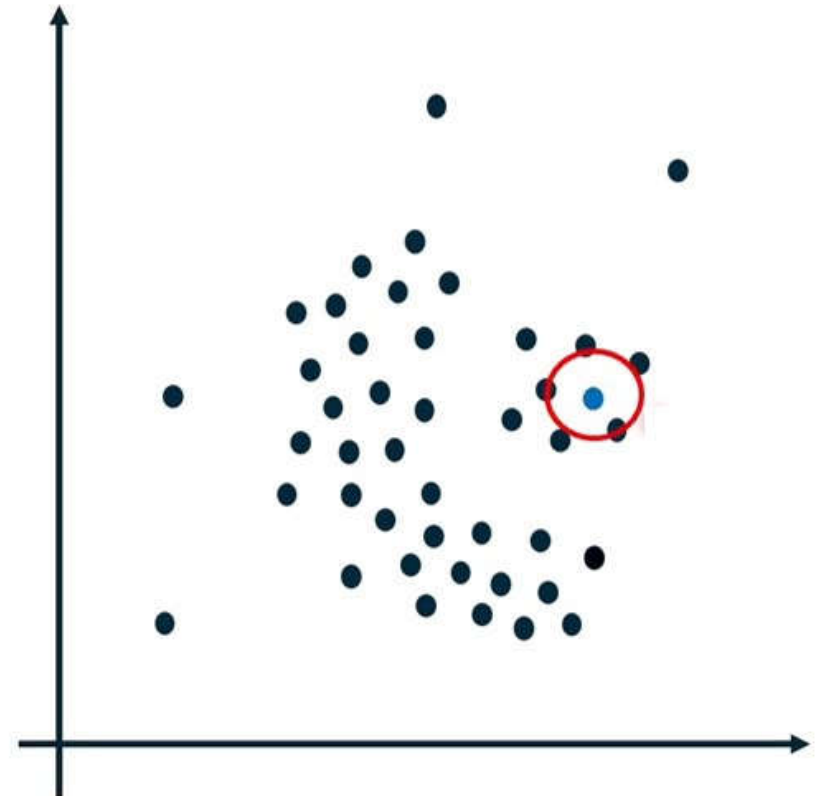
DBSCAN

- The first step in DBSCAN is to classify each data point as a core, border, or noise point.
- To do that, DBSCAN uses ϵ for the distance and min_samples for the minimum number of data points
- For this example, ϵ will be represented by the red circle and min_sample will be 4.
- Note: min_samples includes the point currently being classified.



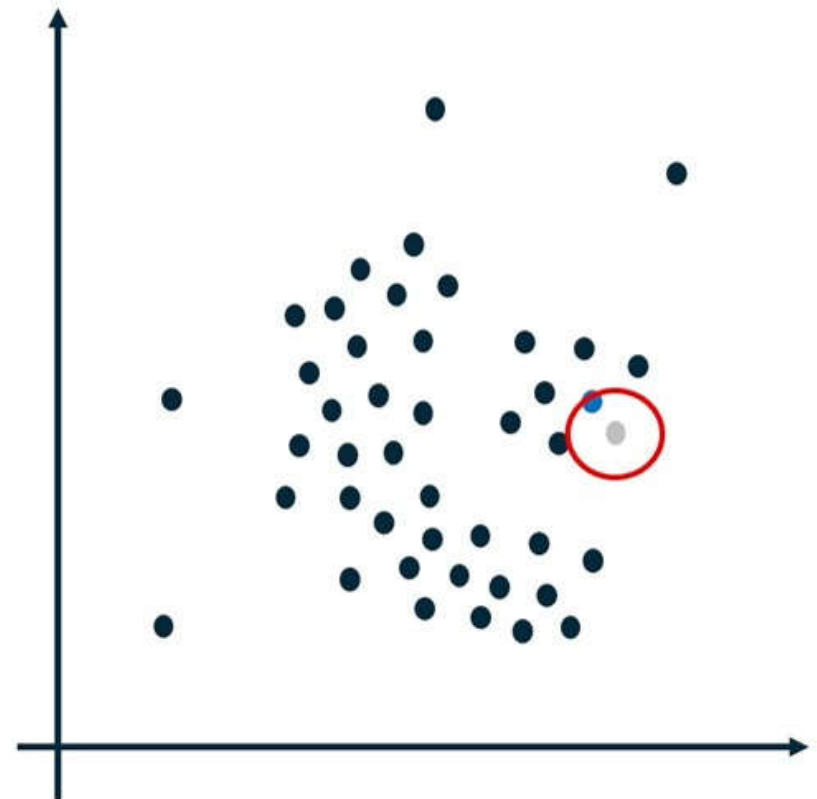
DBSCAN

- The DBSCAN algorithm inspects each data point in the dataset.
- For brevity, we'll only consider a few points.
- Let's say the next point to be evaluated is circled.
- The total number of points within, or contacted by the eps circle is 5.
- This data point is **core**. Core points will be designated by blue.



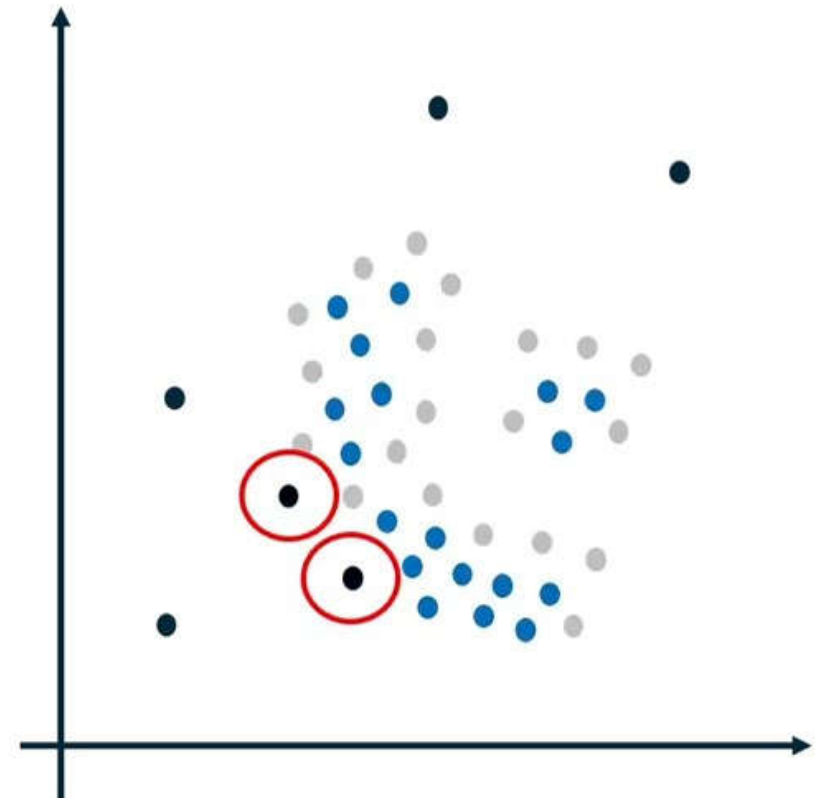
DBSCAN

- The next data point to be classified is in the center of eps circle.
- The total number of points within , or touched by, the eps circle is 3.
- This point is not core but within eps distance of a core point. Therefore, it is a **border** point.
- Border point will be designated by gray.



DBSCAN

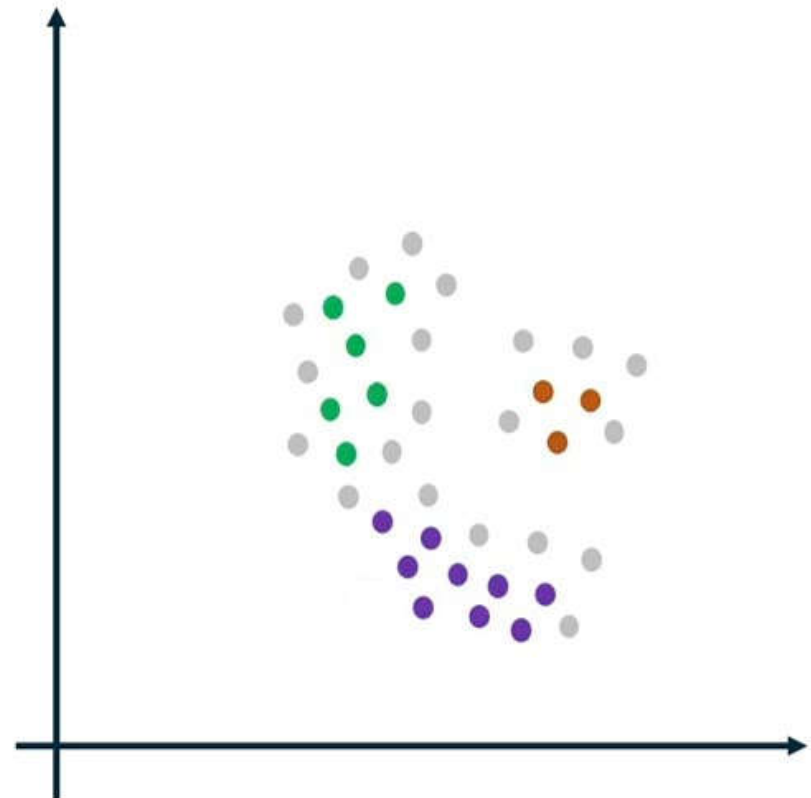
- DBSCAN continues the process with all the data points in the dataset.
- What is depicted here are the results of applying the DBSCAN to the dataset.
- Notice that two data points in addition to the four obvious noise points were identified.





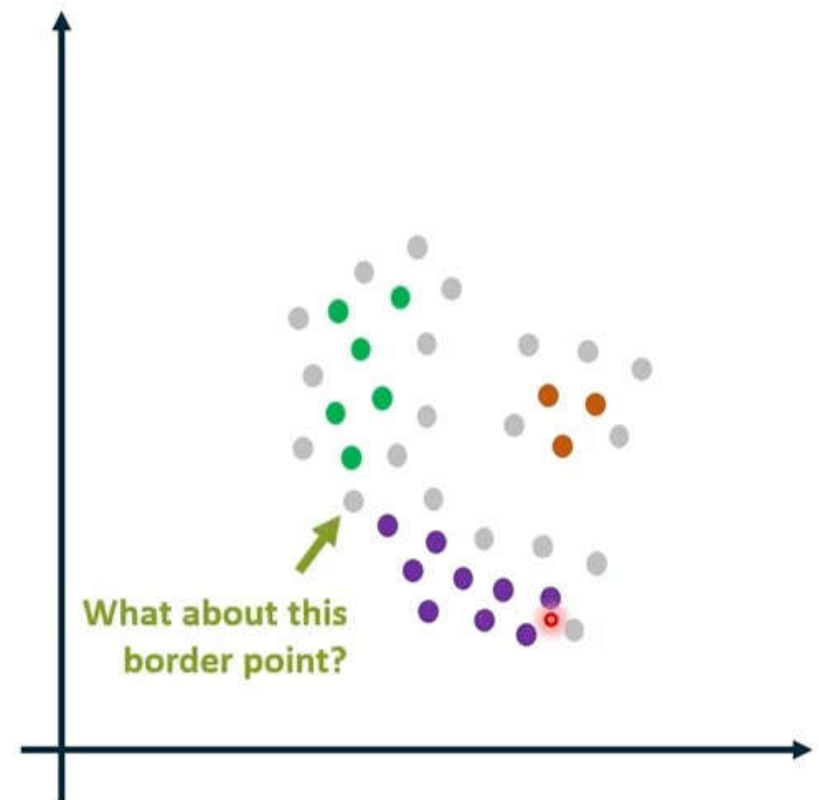
DBSCAN

- Next, DBSCAN eliminates noise points and then connects all core points with ϵ distance of each other.
- Groups of connected core points become clusters.



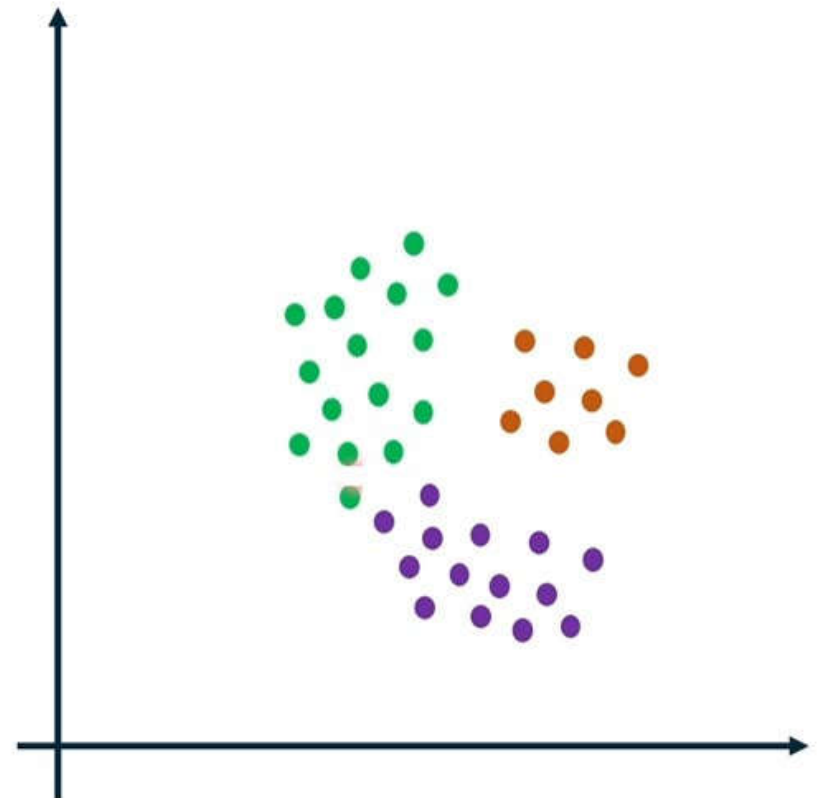
DBSCAN

- Last, DBSCAN assigns border points to clusters based on core point proximity.
- How does DBSCAN handle “ties”?
- DBSCAN is greedy. When necessary, border points are assigned to the cluster that was created first.
- Let’s say that the green cluster was created first.



DBSCAN

- DBSCAN has completed the clustering.
- This algorithm works well if all the clusters are dense enough, and they are well separated by low-density regions.





DBSCAN

- The DBSCAN class in Scikit-Learn is as simple to use as you might expect. Let's test it on the moons dataset:

```
from sklearn.cluster import DBSCAN
from sklearn.datasets import make_moons

X, y = make_moons(n_samples=1000, noise=0.05)
dbscan = DBSCAN(eps=0.05, min_samples=5)
dbscan.fit(X)
```

- The labels of all the instances are available in the **labels_** instance variable:

```
>>> dbscan.labels_
array([ 0, 2, -1, -1, 1, 0, 0, 0, ..., 3, 2, 3, 3, 4, 2, 6, 3])
```



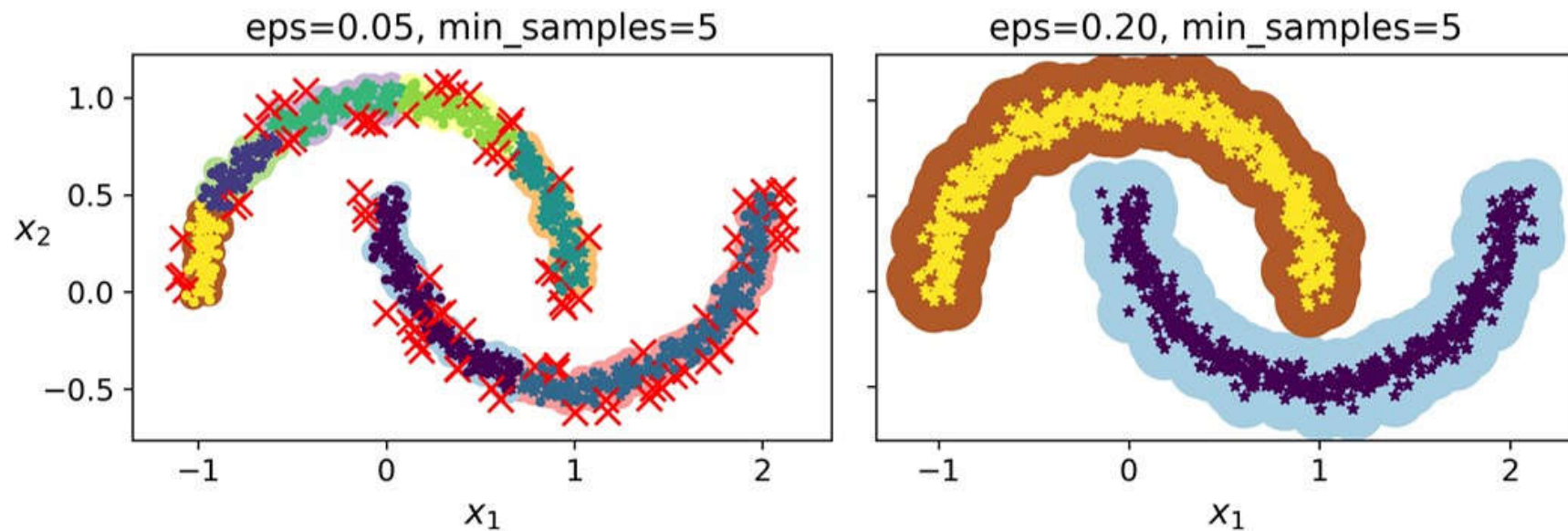
DBSCAN

- Notice that some instances have a cluster index equal to **-1**: this means that they are considered as **anomalies** by the algorithm.
- The indices of the core instances are available in the **core_sample_indices_** instance variable, and the core instances themselves are available in the **components_** instance variable:

```
>>> len(dbscan.core_sample_indices_)
808
>>> dbscan.core_sample_indices_
array([ 0, 4, 5, 6, 7, 8, 10, 11, ..., 992, 993, 995, 997, 998,
       999])
>>> dbscan.components_
array([[ -0.02137124,  0.40618608],
       ...,
       [-0.94355873,  0.3278936 ],
       [ 0.79419406,  0.60777171]])
```

DBSCAN

- If we widen each instance's neighborhood by increasing eps to 0.2, we get the clustering on the right, which looks perfect.



DBSCAN clustering using two different neighborhood radiuses



Thank You